

Complete.PIF.Analysis.partThree.cluster_documentation

Fred Cudhea

2025-05-19

This is documentation for "part 3" of the CRA code, which is for calculating the joint PIFs (attributable mort/incidence of multiple concurrent counterfactual exposures).

First, load libraries, remove functions not needed, and read in source code that defines functions for summing.

```
# Complete.PAF.Analysis.partThree.r

options(digits=20)
library(magrittr)

# debug
# remove all functions for now
rm(list = c("Sum.by.overall", "Sum.by.strata", "Sum.by.strata.joint",
            "sum.stats.maker", "sum.stats.maker.RE.PAFs", "get.RE.PAFs", "get.RE.PAFs.draws"))

source(paste0(code_location, "/debug/Sum_functions_debug_041824.R"))
```

Next, some obsolete code: a function to capitalize every first letter of a word (Used to be used to convert variable names from lower case to upper case. No longer needed since variable names should be consistent at this point in the code now).

```
# MISCELLANEOUS FUNCTIONS -----

# Capitlizes every first letter of a word
capwords <- function(s, strict = FALSE) {
  cap <- function(s) paste(toupper(substring(s, 1, 1)),
                           {s <- substring(s, 2); if(strict) tolower(s) else s},
                           sep = "", collapse = " ")
  sapply(strsplit(s, split = " "), cap, USE.NAMES = !is.null(names(s)))
}
```

Define covar.vec.string (string vector of covariates of interest) so we read in the correct file (defining n.covar is obsolete).

```
#need to redefine covar.vec.string and n.covar since it changes when we do the joint PIFs (since the code for joint PAFs is for
#possible covar.vec.strings)
covar.vec.string<-paste(covar.vec, sep="", collapse="")
n.covar<-length(covar.vec)
if(identical(covar.vec, c("Overall")))
  n.covar<-0

# PAF.draws<-read.csv(file=paste(output_location, diet_pattern, "/all.PIFs_", year.vec.string, "_", covar.vec.string, "_", diet
#                               header=T)
```

Next, we'll define "calc.joint.PAFs", the function that is actually going to calculate the joint PIFs. It reads in a PIF simulations for a given dietary pattern, and calculates joint PIFs for the defined strata of interest. We'll break it down chunk by chunk.

```
# calculates joint PAFs by specified strata (but should only choose the fully stratified option)

# debug
# strata_var=strata_list[[e]]
# strata_var

calc.joint.PAFs <- function(PAF.draws, strata_var)
{
```

Define number of sims (by counting up number of variables that start with "V")

```
n.draws<-length(grep(pattern="V", x=names(PAF.draws)))
#PAF.draws<-PAF.draws[as.character(PAF.draws$riskfactor)!="pufa_c" & as.character(PAF.draws$riskfactor)!="sfat",]
```

Create variable that defines the subgroups of interest based on the strata you selected (If you selected age, sex, and race as your strata of interest, this will make an age/sex/race variable).

```
PAF.draws$strata<-interaction(PAF.draws[,strata_var]) # this should include all vars including outcome and disease_type
```

Split PAF.draws into a list of data frames, with each element being a data frame for a specific subgroup. Since the PIF simulation you read in is specific to the strata you are interested in, each row corresponds to a specific exposure here.

```
PAF.draws.by.strata.outcome<-split(x=PAF.draws, f=list(PAF.draws$strata))
```

Create joint.PAFs.all.draws, a data frame to store all the simulated joint PIFs we are about to calculate. We also identify the column indices for where the simulation values start and end in PAF.draws.

```
joint.PAFs.all.draws<-matrix(nrow=length(PAF.draws.by.strata.outcome), ncol=(length(strata)+n.draws)) #adding plus 1 for pat
joint.PAFs.all.draws<-as.data.frame(joint.PAFs.all.draws)
names(joint.PAFs.all.draws)[1:(length(strata)+n.draws)]<-c(strata, paste("V", 1:n.draws, sep=""))

start<-which(names(PAF.draws.by.strata.outcome[[1]])=="V1")
end<-start+n.draws-1
```

Loop through the subgroups in PAF.draws.by.strata.outcome and, for each subgroup, apply the joint PIF formula to calculate joint PIF for each simulation.

```
for(i in 1:length(PAF.draws.by.strata.outcome))
{
  joint.PAFs.all.draws[i,1:(length(strata))]<-PAF.draws.by.strata.outcome[[i]][1,strata]
  #joint.PAFs.all.draws[i,length(strata)+1]<-as.character(PAF.draws.by.strata.outcome[[i]][["outcome"]])[1]
  joint.PAFs.all.draws[i,length(strata)+c(1:n.draws)]<-1-apply(1-PAF.draws.by.strata.outcome[[i]][,start:end],
                                                                MARGIN=2, FUN=prod)
}
```

Finally, return the data frame of joint PIFs sims by subgroup

```
return(joint.PAFs.all.draws)
}
```

Next function calculates the attributable mortality/incidence to the shift of multiple exposures. Basically just taking in the joint PIF sims and multiplying by mort/incidence sims. strata_var is an input to the function so the function knows what variables to keep for labeling purposes. The inputs, both joint PIFs draws and mort/incidence draws, should already be stratified by your strata of interest.

```
# calculates joint mort, by multiplying joint paf with total mort
calc.joint.mort <- function(joint.paf.draws, total.mort.draws, strata_var)
{
  # joining by age, female, race, outcome
  merged <- merge(joint.paf.draws, total.mort.draws)

  Joint.PAF.starting.point<-which(names(merged=="V1")
  Joint.PAF.ending.point<-which(names(merged)==paste("V", nsim1, sep=""))
  totalmort.starting.point<-which(names(merged=="X1")
  totalmort.ending.point<-which(names(merged)==paste("X", nsim1, sep=""))

  joint.mort<-cbind(merged[,c(strata_var, "outcome")], # this is correct (merge by outcome)
                  merged[,Joint.PAF.starting.point:Joint.PAF.ending.point]*merged[,totalmort.starting.point:totalmort.ending
  return(joint.mort)
}
```

Calculating the joint PIFs are straight forward, but mapping those joint PIFs to the correct mortality/incidence numbers is a little bit trickier due to the presence of "disease_type" as potential strata (Basically, when combining strata, you to sum over age/sex/race/outcome/pathway but you don't want to sum over disease_type, which are, by definition, a different kind of way to categorize "outcome", not defined independently of "outcome"). Brooke's solution is the function below: run_Part3, which takes in integer inputs "e" and "z". Based on what you use for "e" and "z", you calculate joint

PIFs and corresponding mort for a particular strata combination. "e" can take on an integer from 1-4, each corresponding to one of the disease types. "z" corresponds to the index number of your strata combination of interest for strata.combos list created in part 2. This function is going to output a list of 4 data frames: 1: summary stats for the joint PIFs, attributable mort etc... 2: joint attributable mort draws 3: joint totalmort draws, 4: joint "reverse engineered" PIF draws.

```
run_Part3 <- function(e, z){
```

Create covar.vec.string (to read in the correct input file for PIF draws) and read in PIF draws for dietary pattern of interest.

```
# need to redefine covar.vec.string and n.covar since it changes when we do the joint PIFs (since the code for joint PAFs is
# possible covar.vec.strings)
covar.vec.string <- paste(covar.vec, sep="", collapse="")
n.covar<-length(covar.vec)
if(identical(covar.vec, c("Overall"))){
  n.covar<-0

PAF.draws <- read.csv(file=paste(output_location, diet_pattern, "/all.PIFs_", year.vec.string, "_", covar.vec.string, "_",
  header=T)
```

We now create 4 string vectors, one for each set of possible set of strata combinations. disease_type, disease_type1, disease_type2, disease_type3 can never be part of the same strata combination, which is why there are four string vectors, one for each disease type. We put them in a list so we can refer to a specific string vector easily.

```
# create new combinations of strata without disease type overlap
strata0 <- c("age", "female", "race", "outcome", "pathway", "disease_type")
strata1 <- c("age", "female", "race", "outcome", "pathway", "disease_type1")
strata2 <- c("age", "female", "race", "outcome", "pathway", "disease_type2")
strata3 <- c("age", "female", "race", "outcome", "pathway", "disease_type3")
strata_list <- list(strata0, strata1, strata2, strata3)

# Once we know which dietary factors we will keep to calculate joint PIFs, we will subset PAF.draws to keep only
# those dietary factors
```

Create lists to store interim results (though I don't think it's actually necessary to store them in lists. Probably legacy code from a previous attempt using for loops).

```
joint.PAFs.all.draws <- list()
total.mort <- list()
joint.mort.all.draws <- list()
```

We then calculate joint PIFs for strata combination specified by "e" (save to csv). So the resulting output stored in joint.PAFs.all.draws[[e]] is sims for fully stratified strata and with a particular disease_type specified.

```
joint.PAFs.all.draws[[e]] <- calc.joint.PAFs(PAF.draws=PAF.draws, strata_var=strata_list[[e]])

write.csv(x=joint.PAFs.all.draws[[e]], file=paste(output_location, diet_pattern, "/joint/joint_PIFs_all_draws_", paste(strat

# this is correct - compared to nov results - should contain all strata vars
```

Next we need to get the same thing total mortality draws for the same strata. We already have fully stratified total mort sims so it's just a matter of reading it in. We have to remove "disease_type" though since it doesn't factor in the labeling of the total mort sims file.

```
# for e=1, i am commenting out setdiff(outcome)
total.mort[[e]] <- read.csv(paste(output_location, diet_pattern, "/total_USmortality_draws_by_",
  strata_list[[e]] %>%
    # setdiff(., "outcome") %>%
    setdiff(., "disease_type") %>%
    setdiff(., "disease_type1") %>%
    setdiff(., "disease_type2") %>%
    setdiff(., "disease_type3") %>%
    paste(collapse = "_"),
  "_", year.vec, "_", diet_pattern, ".csv", sep=""))
```

Now that we have the fully stratified PIFs and fully stratified sims, we can finally calculate (and) fully stratified joint attributable mort/incidence sims.

```

joint.mort.all.draws[[e]] <- calc.joint.mort(joint.paf.draws=joint.PAFs.all.draws[[e]],
                                           total.mort.draws=total.mort[[e]],
                                           strata=strata_list[[e]])

# this is comparable to nov results
write.csv(x=joint.mort.all.draws[[e]], file=paste(output_location, diet_pattern, "/joint/joint_mort_all_draws_", paste(strat

```

Now, using the sum Sum.by.strata.joint function defined in "Sum_functions_debug_041824.R", we sum the joint attributable mort/incidence by strata of interest (specified by "z", the index corresponding to a list of strata in stata.combos.list), and calculate corresponding summary stats, "reverse-engineered" PIFs, and total mort/incidence, all of which we save in csv files.

```

# Calculate combos of interest

# debug
# z=6 # this works with e=1
# z=7 # this works with e=2
# z=8 # this works with e=3
# z=9 # this works with e=4

print(z)
print(strata.combos[[z]])
print("")

sum.stats.byX.joint.attr.mort <- Sum.by.strata.joint(allmort=joint.mort.all.draws[[e]],
                                                    totalmort=total.mort[[e]],
                                                    pop=pop.draws,
                                                    covar=strata.combos[[z]])

write.csv(x=sum.stats.byX.joint.attr.mort[[1]],
          file=paste(output_location, diet_pattern, "/joint/summarystats_joint_attributable_USmortality_", "by_", paste(strat
          row.names=FALSE)

write.csv(x=sum.stats.byX.joint.attr.mort[[2]],
          file=paste(output_location, diet_pattern, "/joint/joint_attributable_USmortality_draws_", "by_", paste(strata.comb
          row.names=FALSE)

write.csv(x=sum.stats.byX.joint.attr.mort[[3]],
          file=paste(output_location, diet_pattern, "/joint/joint_totalmort_draws_", "by_", paste(strata.combos[[z]], sep="
          row.names=FALSE)

write.csv(x=sum.stats.byX.attr.mort[[4]],
          file=paste(output_location, diet_pattern, "/joint/RE_joint_PIFs_draws_", "by_", paste(strata.combos[[z]], sep="
          row.names=FALSE)

print("done")

}

```

Now that the function run_Part3 is defined, we simply use to calculate joint PIF results for specified strata of interest (again, we specify using e and z, rather than a vector of strings). Original intention was to do it for all strata combinations but it took too long (We are calculating the unstratified joint PIFs each time we use run_part3 which might have to do it with it), so we specify which strata combos and get results specifically for those (depending on what we're intending to report for a particular paper).

```
# combos of interest

# run_Part3(e=1, z=1) # age
# run_Part3(e=1, z=2) # sex
# run_Part3(e=1, z=3) # race
# run_Part3(e=1, z=4) # outcome
#
# run_Part3(e=1, z=6) # disease_type
# run_Part3(e=2, z=7) # disease_type1
# run_Part3(e=3, z=8) # disease_type2
# run_Part3(e=4, z=9) # disease_type3

# FOR 4-PILLAR PAPER
run_Part3(e=1, z=10) # Age and Sex
run_Part3(e=1, z=32) # Outcome and Disease_type
run_Part3(e=1, z=36) # Pathway and Disease_type

# FOR HEALTH PAPER (LU)
run_Part3(e=1, z=12) # Age and Outcome
run_Part3(e=2, z=15) # Age and Disease type 1
run_Part3(e=3, z=16) # Age and Disease type 2
run_Part3(e=1, z=19) # Sex and Outcome
run_Part3(e=2, z=22) # Sex and Disease type 1
run_Part3(e=3, z=23) # Sex and Disease type 2
run_Part3(e=1, z=25) # Race and Outcome
run_Part3(e=2, z=28) # Race and Disease type 1
run_Part3(e=3, z=29) # Race and Disease type 2

# DEBUG
# all combos of interest
# for(i in strata.combos.of.interest.index)
# {
#   print(i)
#   print(strata.combos[[i]])
# }

# test
# print(strata.combos[[15]])
```